

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool (BL3)

Assessment Tool Weightage: 20%

QUESTIONS:

Duration :60 Mins
On class
Total Marks:50

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

*I E.Kavya(192424365) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

E.Kavya

Annexure A

ERROR ANALYSIS TASK

Q1. The following C program is intended to perform string processing, function calls, array manipulation, conditional execution, pointer operations, switch-case selection, and loop execution. However, the program contains several errors introduced during different phases of compilation, including lexical, syntax, semantic, logical, and runtime errors.

Given Program

```
#include <stdio.h>
#include <string.h>

int findLength(char str[])
{
    return strlen(str);
}

int main()
{
    char name[20] = "Compiler"

    int count;

    count = findLength();

    if(count > 5
    {
        printf("Length is greater than 5\n");
    }

    int marks[4] = {85, 90, 75, 80};

    printf("%d\n", marks[4]);

    int choice = 2;

    switch(choice)
    {
        case 1:
            printf("Option 1\n");
            break;

        case 2
            printf("Option 2\n");
            break;

        default:
            printf("Invalid Choice\n");
```

```

    }

    int *ptr;
    printf("%d\n", *ptr);

    int total = 100;
    int students = 0;

    float average = total / students;

    int i = 1;

    while(i <= 5)
    {
        printf("%d\n", i);
    }

    return 0;
}

```

Tasks

- Identify all errors in the program. Mention the line number, error type, and reason for each error. (5 Marks)
- Classify the identified errors as Lexical, Syntax, Semantic, Logical, or Runtime errors in a suitable table. (5 Marks)
- Explain at which phase of compilation each category of error is detected. Also mention which errors are detected only during execution. (4 Marks)
- Rewrite the program after correcting all the errors so that it executes successfully. (4 Marks)
- Suggest any three compiler features or warnings that can help detect such programming errors before execution. (2 Marks)

Q2. The following C program is intended to perform function calls, array processing, conditional execution, switch-case selection, and loop operations. However, several syntax errors have been intentionally introduced by violating the grammatical rules of the C language. These errors include missing delimiters, unmatched parentheses, incorrect control structures, malformed function definitions, and incomplete statements. Such errors are normally detected during the syntax analysis (parsing) phase of a compiler.

Carefully examine the program and answer the questions that follow.

Given Program

```
#include <stdio.h>
```

```

int maximum(int x, int y
{
    if(x > y)
        return x;
    else
        return y;
}

int main()
{
    int marks[5] = {70, 80, 90, 85, 95};

    int max = maximum(marks[0], marks[1]);

    switch(max)
    {
        case 80:
            printf("Maximum is 80\n")
            break;

        case 90
            printf("Maximum is 90\n");
            break;

        default:
            printf("Other Value\n");
    }

    do
    {
        printf("%d\n", max);
        max--;
    } while(max > 75);

    if(max == 75)
        printf("Limit Reached\n")
    }

    return 0;

```

Tasks

- (a) Identify all syntax errors in the program. Mention the line number and reason for each error. (4 Marks)
- (b) Classify the identified syntax errors under the appropriate categories and present them in a table. (3 Marks)
- (c) Correct the syntax errors and rewrite the complete program. (4 Marks)
- (d) Explain how the parser detects syntax errors and how error recovery enables further parsing. (2 Marks)
- (e) Suggest any two improvements that can help a compiler generate better syntax error messages. (2 Marks)

Q3. The following C program is intended to perform factorial computation, function calls, array processing, pointer manipulation, conditional execution, switch-case selection, and loop operations. However, the program contains several errors introduced during different phases of compilation and execution, including lexical, syntax, semantic, intermediate code generation, code optimization, and runtime errors.

Carefully examine the program and answer the questions that follow.

Given Program

```
#include <stdio.h>
```

```
int factorial(int n)
```

```
{
```

```
    if(n <= 1)
```

```
        return 1;
```

```
    return n * factorial(n - 1);
```

```
}
```

```
int main()
```

```
{
```

```
    int @num = 5;
```

```
    int value = 10
```

```
    int result;
```

```
    result = factorial();
```

```
if(result >= 100)
{
    printf("Large Number\n");
}

switch(result)
{
    case 24:
        printf("Factorial of 4\n");
        break;

    case 120
        printf("Factorial of 5\n");
        break;

    default:
        printf("Other Value\n");
}

int numbers[5] = {2,4,6,8,10};
printf("%d\n", numbers[5]);

int *ptr;
printf("%d\n", *ptr);

int a = 50;
int b = 0;
int c = a / b;

int sum = result + 0 + value;

int i = 1;
while(i <= 5)
{
    printf("%d\n", i);
}

return 0;
}
```

Tasks

(a) Identify all the errors in the program. Mention the line number, error type, and reason for each error. (4 Marks)

(b) Classify the identified errors according to the appropriate compiler phase (Lexical, Syntax, Semantic, Intermediate Code Generation, Code Optimization, and Runtime) using a suitable table. (4 Marks)

(c) Explain how the compiler processes the program by identifying the errors detected during each compilation phase and those detected only at runtime. (3 Marks)

(d) Rewrite the complete corrected program so that it compiles and executes without errors. (2 Marks)

(e) Suggest any two improvements in compiler design or optimization techniques that could detect or prevent such errors before program execution. (2 Marks)

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Solutions

23/7/26

1.(a) Identify all errors in the program

Line No.	Error	Error Type	Reason
12	char name[20] = "Compiler"	Syntax Error	Missing semicolon (;).
16	count = findLength();	Semantic Error	Function requires one argument (name).
18	if(count > 5	Syntax Error	Missing closing parenthesis).
27	printf("%d\n", marks[4]);	Runtime Error	Array index 4 is out of bounds (valid index: 0–3).
35	case 2	Syntax Error	Missing colon (:).
45	printf("%d\n", *ptr);	Runtime Error	Pointer is uninitialized before dereferencing.
50	float average = total / students;	Runtime Error	Division by zero.
54	while(i <= 5)	Logical Error	Variable i is never incremented, causing an infinite loop.

1.(b) Classification of Errors

Error	Category
Missing semicolon after "Compiler"	Syntax Error
Missing function argument	Semantic Error
Missing) in if statement	Syntax Error
Missing : after case 2	Syntax Error
Accessing marks[4]	Runtime Error
Dereferencing uninitialized pointer	Runtime Error
Division by zero	Runtime Error
Infinite while loop	Logical Error

Lexical Error: None in this program.

1.(c) Compilation Phases

Error Category	Detected During
Lexical Error	Lexical Analysis Phase
Syntax Error	Syntax Analysis (Parsing) Phase
Semantic Error	Semantic Analysis Phase
Logical Error	Not detected by compiler; found during testing/program execution
Runtime Error	Detected only during program execution

Explanation

- **Lexical Analysis**
 - Checks tokens, identifiers, keywords, operators.
 - Example: Invalid symbol (@, \$, etc.).
- **Syntax Analysis**
 - Checks grammar rules.
 - Example: Missing ;,), ::.
- **Semantic Analysis**
 - Checks meaning of statements.
 - Example: Wrong function arguments.
- **Runtime**
 - Errors occur only when executing.
 - Example: Division by zero, invalid pointer, array out of bounds.
- **Logical**
 - Program compiles and runs but produces wrong output.
 - Example: Infinite loop.

1.(d) Corrected Program

```
#include <stdio.h>
#include <string.h>
int findLength(char str[])
{
```

```

    return strlen(str);
}
int main()
{
    char name[20] = "Compiler";
    int count;
    count = findLength(name);
    if(count > 5)
    {
        printf("Length is greater than 5\n");
    }
    int marks[4] = {85, 90, 75, 80};
    printf("%d\n", marks[3]);
    int choice = 2;
    switch(choice)
    {
        case 1:
            printf("Option 1\n");
            break;
        case 2:
            printf("Option 2\n");
            break;
        default:
            printf("Invalid Choice\n");
    }
    int value = 50;
    int *ptr = &value;
    printf("%d\n", *ptr);
    int total = 100;
    int students = 5;
    float average = (float)total / students;
    printf("Average = %.2f\n", average);
    int i = 1;
    while(i <= 5)
    {
        printf("%d\n", i);
        i++;
    }
}

```

```

    }
    return 0;
}

```

```

Output
Length is greater than 5
80
Option 2
50
Average = 20.00
1
2
3
4
5

```

1.(e) Three Compiler Features/Warnings

1. -Wall
 1. Enables common compiler warning messages.
2. -Wextra
 1. Detects additional programming mistakes not covered by -Wall.
3. -Wpedantic
 1. Checks whether the code follows the C language standard strictly.

Example compilation command (GCC):

`gcc program.c -Wall -Wextra -Wpedantic -o program` These options help detect many mistakes such as missing function arguments, unused variables, type mismatches, and other potential bugs before execution.

2.(a) Identify all syntax errors in the program

Line No.	Syntax Error	Reason
3	int maximum(int x, int y	Missing closing parenthesis) before {.

Line No.	Syntax Error	Reason
18	printf("Maximum is 80\n")	Missing semicolon (;).
21	case 90	Missing colon (:).
33	while(max > 75);	Missing closing brace } before while in do-while loop.
35	printf("Limit Reached\n"	Missing closing parenthesis) and semicolon (;).
36	}	Extra closing brace due to previous syntax error.
38	return 0;	Should be inside main() function.
39	End of program	Missing closing brace } for main() function.

2.(b) Classification of Syntax Errors

Error	Category
Missing) in function definition	Missing Delimiter
Missing ; after printf()	Missing Statement Terminator
Missing : after case 90	Malformed switch-case Statement
Missing } before while	Incorrect Loop Structure
Missing) and ; in printf()	Incomplete Statement
Missing closing brace for main()	Unmatched Braces

2.(c) Corrected Program

```

#include <stdio.h>
int maximum(int x, int y)
{
    if(x > y)
        return x;
    else
        return y;
}
int main()
{
    int marks[5] = {70, 80, 90, 85, 95};

```

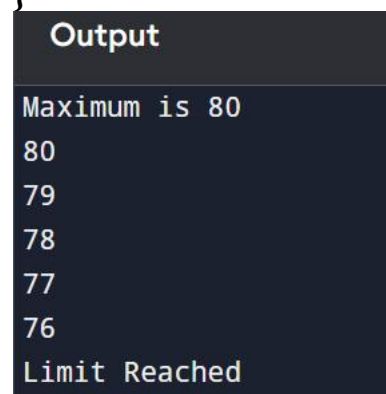
```

int max = maximum(marks[0], marks[1]);

switch(max)
{
    case 80:
        printf("Maximum is 80\n");
        break;
    case 90:
        printf("Maximum is 90\n");
        break;

    default:
        printf("Other Value\n");
}
do
{
    printf("%d\n", max);
    max--;
}
while(max > 75);
if(max == 75)
{
    printf("Limit Reached\n");
}
return 0;
}

```



Output

```

Maximum is 80
80
79
78
77
76
Limit Reached

```

2.(d) How the Parser Detects Syntax Errors and Error Recovery

Parser Detection

- The **parser (syntax analyzer)** checks whether the program follows the grammar rules of the C language.

- It verifies the correct arrangement of tokens such as keywords, identifiers, operators, delimiters, and control structures.
- If the parser finds a violation (missing ;,), }, : etc.), it reports a syntax error.

Error Recovery

Error recovery allows the compiler to continue parsing after detecting an error instead of stopping immediately.

Common techniques include:

- **Panic Mode Recovery:** Skips input until a valid symbol (such as ; or }) is found.
- **Phrase-Level Recovery:** Makes small corrections like inserting or deleting a missing token to continue parsing.

2.(e) Two Improvements for Better Syntax Error Messages

1. Precise Line and Column Numbers

- Display the exact location of the syntax error to help programmers identify it quickly.

2. Meaningful Error Suggestions

- Provide corrective messages such as:
 - *"Expected ';' after statement."*
 - *"Missing ')' before '{'."*

These improvements make debugging faster and help programmers correct syntax errors more easily.

3.(a) Identify all the errors in the program

Line No.	Error	Error Type	Reason
13	int @num = 5;	Lexical Error	@ is an invalid character in a C identifier.
15	int value = 10	Syntax Error	Missing semicolon (;).

Line No.	Error	Error Type	Reason
19	result = factorial();	Semantic Error	Function factorial() requires one argument.
31	case 120	Syntax Error	Missing colon (:).
39	printf("%d\n", numbers[5]);	Runtime Error	Array index 5 is out of bounds (valid indices are 0–4).
42	printf("%d\n", *ptr);	Runtime Error	Pointer ptr is uninitialized before dereferencing.
46	int c = a / b;	Runtime Error	Division by zero.
48	int sum = result + 0 + value;	Code Optimization	Adding 0 is a redundant operation that an optimizer can eliminate.
51–54	while(i <= 5)	Runtime / Logical Error	i is never incremented, causing an infinite loop.

3.(b) Classification According to Compiler Phase

Error	Compiler Phase
Invalid identifier @num	Lexical Analysis
Missing ; after value = 10	Syntax Analysis
Missing function argument in factorial()	Semantic Analysis
Missing : after case 120	Syntax Analysis
result + 0 (redundant addition)	Code Optimization
Array index out of bounds (numbers[5])	Runtime
Dereferencing uninitialized pointer	Runtime
Division by zero	Runtime
Infinite while loop	Runtime / Logical

3.(c) Compiler Phases and Error Detection

Phase	Errors Detected
Lexical Analysis	Detects invalid tokens such as @num.
Syntax Analysis	Detects missing semicolons, parentheses, braces, colons, etc.

Phase	Errors Detected
Semantic Analysis	Detects incorrect function calls, type mismatches, undeclared variables, etc.
Intermediate Code Generation	Converts the validated program into an intermediate representation (IR). It assumes lexical, syntax, and semantic errors have been corrected.
Code Optimization	Removes unnecessary computations such as result + 0.
Runtime	Detects errors that occur only during execution, such as division by zero, invalid array access, uninitialized pointers, and infinite loops.

3.(d) Corrected Program

```

#include <stdio.h>
int factorial(int n)
{
    if(n <= 1)
        return 1;
    return n * factorial(n - 1);
}
int main()
{
    int num = 5;
    int value = 10;
    int result;

    result = factorial(num);
    if(result >= 100)
    {
        printf("Large Number\n");
    }
    switch(result)
    {
        case 24:
            printf("Factorial of 4\n");
            break;
        case 120:
            printf("Factorial of 5\n");
            break;
        default:

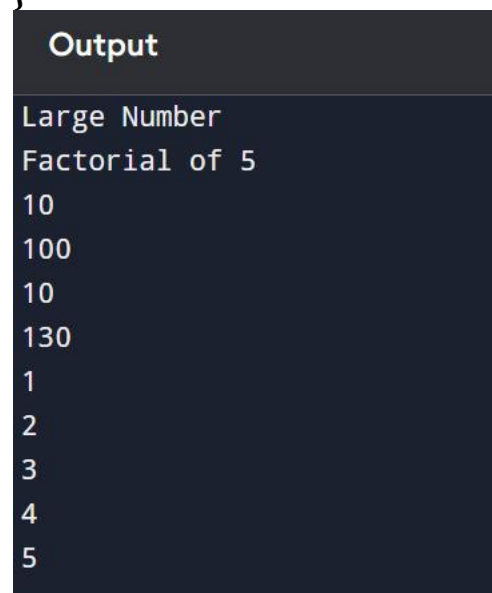
```



```

        printf("Other Value\n");
    }
    int numbers[5] = {2,4,6,8,10};
    printf("%d\n", numbers[4]);
    int val = 100;
    int *ptr = &val;
    printf("%d\n", *ptr);
    int a = 50;
    int b = 5;
    int c = a / b;
    printf("%d\n", c);
    int sum = result + value;
    printf("%d\n", sum);
    int i = 1;
    while(i <= 5)
    {
        printf("%d\n", i);
        i++;
    }
    return 0;
}

```



```

Output
Large Number
Factorial of 5
10
100
10
130
1
2
3
4
5

```

3.(e) Two Improvements in Compiler Design / Optimization

1. Enable Compiler Warnings

1. Compile with options such as -Wall and -Wextra to detect missing arguments, uninitialized variables, and suspicious code.

2. Static Analysis and Optimization

1. Use static analysis tools and compiler optimizations to identify issues such as redundant expressions (+0), possible division by zero, and potential array out-of-bounds access before execution.